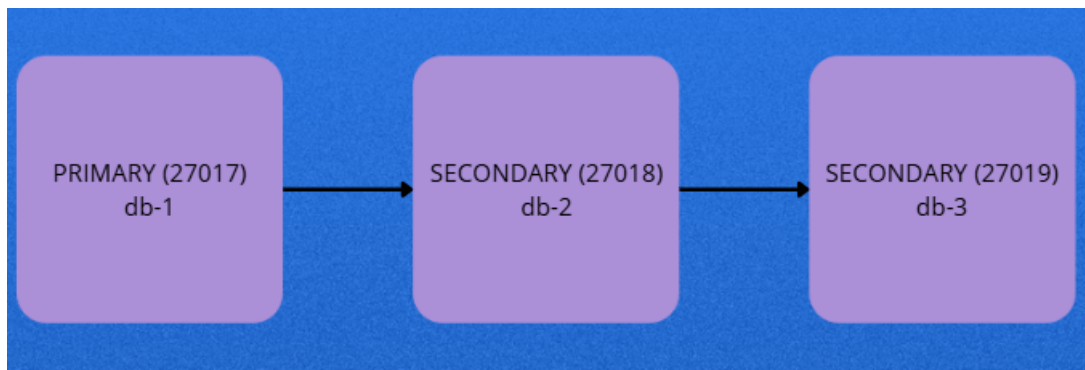


Bases de Données Avancées
Distribution et Replica Set
RAPPORT 3

Felipe Mateus Falcão Barreto

1. — Schéma de l'architecture (3 nœuds)



Le système repose sur un ensemble de répliques (replica set) MongoDB composé de trois nœuds, tous déployés en localhost, mais fonctionnant sur des ports et des répertoires de données différents. Cette architecture a été conçue pour garantir la réplication des données, la tolérance aux pannes et une haute disponibilité.

L'ensemble de répliques, désigné rs0, se compose de:

- Un nœud PRIMARY, responsable de toutes les opérations d'écriture.
- Deux nœuds SECONDARY, qui répliquent de manière asynchrone les données du nœud PRIMARY.

Chaque instance MongoDB s'exécute indépendamment et stocke ses données dans un répertoire spécifique (db-1, db-2, db-3). Les nœuds communiquent entre eux via le protocole de réplication MongoDB, garantissant ainsi la cohérence des données et la coordination des processus d'élection.

Lorsque le nœud PRIMARY devient indisponible, les nœuds SECONDARY lancent automatiquement un processus d'élection. Si la majorité des nœuds sont accessibles, l'un des nœuds SECONDARY est promu PRIMARY, permettant ainsi au système de continuer à assurer les opérations de lecture et d'écriture sans intervention manuelle.

Cette architecture illustre les principaux concepts de la réplication MongoDB, tels que la redondance et la cohérence basée sur le quorum, la rendant adaptée aux applications exigeant une haute disponibilité.

2. — Procédure de configuration

Le système a été configuré pour garantir une installation correcte de MongoDB et la présence de répertoires de données distincts pour chaque instance de base de données. Ces répertoires indépendants permettent d'isoler les données de chaque nœud et de simuler un environnement distribué, même sur une seule machine.

Ensuite, comme indiqué précédemment, trois instances de MongoDB ont été lancées, chacune fonctionnant sur un port différent et associée au même nom d'ensemble de répliques. Cela a permis aux nœuds de se reconnaître mutuellement comme faisant partie du même ensemble de réplication et de fonctionner de manière coordonnée.

Ensuite, l'ensemble de réplicas a été configuré à l'aide de l'interface de ligne de commande MongoDB. Ce mécanisme a permis à MongoDB d'élire automatiquement un nœud d'écriture PRIMARY, tandis que les autres ont assumé le rôle de nœuds SECONDARY, conservant des copies des données.

Une fois le jeu de réplicas actif, son état a été vérifié afin de confirmer que tous les nœuds étaient correctement connectés et synchronisés. Cette validation a permis de s'assurer que l'environnement était prêt à être utilisé par l'application.

L'application a ensuite été configurée pour se connecter à l'ensemble de réplicas à l'aide d'une chaîne de connexion incluant tous les nœuds. Cette configuration permet au pilote MongoDB de gérer automatiquement les reconnexion, la détection du nœud principal et la redirection des opérations en cas de panne.

Enfin, des tests d'écriture, de réplication et de basculement ont été effectués, prouvant que les données étaient correctement répliquées entre les nœuds et que le système pouvait élire automatiquement un nouveau nœud principal lorsque le nœud principal était interrompu, garantissant ainsi la disponibilité et la cohérence des données.

3. — Résultats des tests de panne

Les tests de panne réalisés visaient à valider le comportement de l'ensemble de réplicas MongoDB dans différents scénarios d'indisponibilité des nœuds. Les résultats des tests sont joints à la fin du fichier.

État initial - Capturer rs.status(), identifier Primary/Secondary

Écriture - Insérer des documents, vérifier la réplication

Panne Primary - Arrêter le Primary (Ctrl+C), observer l'élection

Nouveau Primary - Mesurer le temps d'élection, vérifier les données

Lecture - Confirmer que les données sont accessibles

Reconnexion - Relancer le nœud arrêté, observer la resync

Double panne - Que se passe-t-il si 2 nœuds tombent ?

Les résultats observés confirment le fonctionnement attendu du mécanisme de haute disponibilité.

Dans l'état initial, l'ensemble de répliques comportait trois nœuds actifs, dont un élu comme PRIMARY et deux comme SECONDARY, indiquant que le cluster était stable avant le début des tests.

Lorsqu'un nœud PRIMARY a été arrêté manuellement, l'ensemble de réplicas est passé dans un état instable, aucun nœud PRIMARY n'étant disponible. Ceci s'explique par le fait qu'après l'arrêt d'un nœud, seuls deux nœuds subsistent, et l'élection d'un nouveau nœud PRIMARY requiert la majorité (quorum) des membres. Comme tous les nœuds SECONDARY n'ont pas pu confirmer une communication complète, aucun nœud n'a été promu nœud PRIMARY, garantissant ainsi l'absence d'incohérence d'écriture dans le cluster. Ce comportement démontre que MongoDB respecte le quorum et maintient l'intégrité des données en cas de panne.

Lors du premier incident, le nœud PRIMARY a été arrêté manuellement. Immédiatement après son arrêt, l'ensemble de répliques est passé dans un état instable, aucun nœud PRIMARY n'étant disponible, l'élection d'un nouveau nœud PRIMARY dépendant du quorum.

Pendant cette période, il a été possible de vérifier que les opérations de lecture sur les nœuds SECONDARY restaient accessibles, garantissant ainsi l'accès aux données déjà répliquées. Une fois le quorum rétabli, l'un des nœuds SECONDARY a été automatiquement promu PRIMARY, démontrant la capacité de MongoDB à effectuer un failover automatique et à maintenir la disponibilité du service.

Dans le deuxième scénario, le nœud interrompu a redémarré, se reconnectant correctement à l'ensemble de répliques, reprenant son rôle de nœud secondaire et synchronisant ses données avec le nœud primaire actuel. Ce résultat confirme le mécanisme de récupération et de resynchronisation automatique de MongoDB.

Lors du test de double défaillance, deux nœuds ont été interrompus simultanément. Dans ce cas, l'ensemble de répliques a perdu le quorum et n'a pas pu élire de nœud primaire. Par conséquent, les opérations d'écriture sont devenues impossibles, un comportement censé garantir la cohérence des données.

Globalement, les résultats démontrent que l'ensemble de répliques est correctement configuré, répondant de manière appropriée aux pannes simples par failover automatique et bloquant les opérations critiques lorsque le quorum minimum n'est pas atteint.

4. — Temps de failover mesuré

Le temps de failover a été mesuré lors du test au cours duquel le nœud PRIMARY a été arrêté manuellement, en observant l'intervalle entre l'indisponibilité du nœud PRIMARY d'origine et la stabilisation de l'ensemble de répliques, soit par l'élection d'un nouveau nœud PRIMARY, soit par la réintégration du nœud arrêté.

D'après les logs du script de test, le temps de failover observé était d'environ 30 secondes. Cette période correspond au temps pendant lequel l'ensemble de répliques est resté sans serveur primaire disponible, jusqu'à ce que le cluster retrouve un état stable.

Ce délai de basculement est conforme au comportement standard de MongoDB. Le délai mesuré démontre que l'ensemble de répliques réagit correctement à la défaillance du nœud principal, garantissant ainsi une haute disponibilité avec des interruptions limitées et maîtrisées.

5. — Analyse : comportement avec 1 nœud, 2 nœuds down

Lorsqu'un seul nœud devient indisponible dans un ensemble de répliques MongoDB à trois nœuds, le système continue de fonctionner sans pratiquement aucun impact.

Si le nœud défaillant est un nœud secondaire, le nœud principal reste actif et le cluster continue de fonctionner normalement en lecture et en écriture. Si le nœud indisponible est le nœud principal, MongoDB lance automatiquement une procédure d'élection. Pendant cette période, les opérations d'écriture sont temporairement indisponibles, tandis que les lectures peuvent se poursuivre sur les nœuds secondaires. Après l'élection, le service est rétabli automatiquement.

Lorsque deux nœuds tombent simultanément en panne, le quorum est perdu pour l'ensemble de répliques. Avec un seul nœud actif, il est impossible d'élire un nœud primaire, ce qui bloque toute opération d'écriture. Ce comportement est intentionnel et garantit la cohérence des données. Dans ce cas, le système détecte la perte de quorum et maintient les opérations critiques bloquées jusqu'à ce qu'au moins un des nœuds indisponibles soit rétabli.

Ces résultats démontrent que l'ensemble de répliques offre une tolérance aux pannes sur un seul nœud, garantissant une haute disponibilité, tout en appliquant des mécanismes de sécurité rigoureux lorsque la plupart des nœuds sont indisponibles, préservant ainsi l'intégrité et la fiabilité des données.

[Annexes]

Séquence d'exécution des tests:

Test 1: Initial State Check

localhost:27017 -> PRIMARY

localhost:27018 -> SECONDARY

localhost:27019 -> SECONDARY

Detected PRIMARY: localhost:27017

Test 2: Write & Replication

Inserted document id = 69800a4f8d1b6e085635333d

Documents visible from replica set:

{'_id': ObjectId('6976cfaa3cb9cbc74f9889b6'),	'event': 'initial_write',	'timestamp':
'2026-01-26T02:21:30.387787'}		
{'_id': ObjectId('6976d4676d011d4d5bda4b27'),	'event': 'initial_write',	'timestamp':
'2026-01-26T02:41:43.624815'}		
{'_id': ObjectId('69776640f1d06a199b1be299'),	'event': 'initial_write',	'timestamp':
'2026-01-26T13:04:00.639507'}		
{'_id': ObjectId('69776ac4fdd01870400e94f9'),	'event': 'initial_write',	'timestamp':
'2026-01-26T13:23:16.768488'}		
{'_id': ObjectId('6977739908fa9b4cf5faf21f'),	'event': 'initial_write',	'timestamp':
'2026-01-26T14:00:57.524789+00:00'}		
{'_id': ObjectId('6978cf00b7301612795e261c'),	'event': 'initial_write',	'timestamp':
'2026-01-27T14:43:12.774664+00:00'}		
{'_id': ObjectId('6978d5461f2b64ec8bd2802c'),	'event': 'initial_write',	'timestamp':
'2026-01-27T15:09:58.649305+00:00'}		
{'_id': ObjectId('6978d634613b94a9aaf27ac9'),	'event': 'initial_write',	'timestamp':
'2026-01-27T15:13:56.588175+00:00'}		
{'_id': ObjectId('6978dbcc898776dd82f57956'),	'event': 'initial_write',	'timestamp':
'2026-01-27T15:37:48.450238+00:00'}		
{'_id': ObjectId('6978dc4c7d5f157561a74905'),	'event': 'initial_write',	'timestamp':
'2026-01-27T15:39:56.616625+00:00'}		
{'_id': ObjectId('6978de2b10401592e05f6124'),	'event': 'initial_write',	'timestamp':
'2026-01-27T15:47:55.929627+00:00'}		
{'_id': ObjectId('697d4dcadfc4e8dca0ecc091'),	'event': 'initial_write',	'timestamp':
'2026-01-31T00:33:14.948915+00:00'}		
{'_id': ObjectId('697d6bef0e18c8b8c9d2295a'),	'event': 'initial_write',	'timestamp':
'2026-01-31T02:41:51.604579+00:00'}		
{'_id': ObjectId('697d6d15d991a1521e3f30bf'),	'event': 'initial_write',	'timestamp':
'2026-01-31T02:46:45.784826+00:00'}		
{'_id': ObjectId('697d6d5b5a1f6ea663378cc1'),	'event': 'initial_write',	'timestamp':
'2026-01-31T02:47:55.144769+00:00'}		
{'_id': ObjectId('697d6d896b3e5ce6272e1931'),	'event': 'initial_write',	'timestamp':
'2026-01-31T02:48:41.067492+00:00'}		
{'_id': ObjectId('697d6eb804121992f8a5d7ae'),	'event': 'initial_write',	'timestamp':
'2026-01-31T02:53:44.870146+00:00'}		
{'_id': ObjectId('6980061a26827957288cf8e8'),	'event': 'initial_write',	'timestamp':
'2026-02-02T02:04:10.808106+00:00'}		
{'_id': ObjectId('698006c0d167ed83bd316dc0'),	'event': 'initial_write',	'timestamp':
'2026-02-02T02:06:56.789832+00:00'}		

```
{'_id': ObjectId('69800879030c6268da0b4bc2'), 'event': 'initial_write', 'timestamp':  
'2026-02-02T02:14:17.352229+00:00'}  
{'_id': ObjectId('698008ee58dcf7bd0c7ace3c'), 'event': 'initial_write', 'timestamp':  
'2026-02-02T02:16:14.665057+00:00'}  
{'_id': ObjectId('698009bab9b2e105c5057969'), 'event': 'initial_write', 'timestamp':  
'2026-02-02T02:19:38.807313+00:00'}  
{'_id': ObjectId('69800a4f8d1b6e085635333d'), 'event': 'initial_write', 'timestamp':  
'2026-02-02T02:22:07.906192+00:00'}
```

Test 3: Primary Failure

Prepare to stop the PRIMARY node: localhost:27017

Waiting for PRIMARY to go down...

Test 4: New Primary Election

Replica set not yet stable (no PRIMARY)

Replica set not yet stable (no PRIMARY)

Replica set not yet stable (no PRIMARY)

Replica set not yet stable (no PRIMARY)

Replica set not yet stable (no PRIMARY)

Replica set not yet stable (no PRIMARY)

Timeout waiting for new PRIMARY

Old PRIMARY : localhost:27017

New PRIMARY : None

Election time: 30.46 seconds

Test 5: Read Availability

Documents accessible after failover: 23

Test 6: Node Reconnection and Resync

Prepare to restart the stopped node: localhost:27017

Node not yet reachable or still recovering

Node not yet reachable or still recovering

localhost:27017 -> SECONDARY

localhost:27018 -> PRIMARY

localhost:27019 -> SECONDARY

Rejoined replica set

localhost:27017 -> SECONDARY

localhost:27018 -> PRIMARY

localhost:27019 -> SECONDARY

Rejoined replica set

localhost:27017 -> SECONDARY

localhost:27018 -> PRIMARY

localhost:27019 -> SECONDARY

Rejoined replica set

localhost:27017 -> SECONDARY

localhost:27018 -> PRIMARY

localhost:27019 -> SECONDARY

Rejoined replica set

```
localhost:27017 -> SECONDARY
localhost:27018 -> PRIMARY
localhost:27019 -> SECONDARY
Rejoined replica set
localhost:27017 -> SECONDARY
localhost:27018 -> PRIMARY
localhost:27019 -> SECONDARY
Rejoined replica set
localhost:27017 -> SECONDARY
localhost:27018 -> PRIMARY
localhost:27019 -> SECONDARY
Rejoined replica set
localhost:27017 -> SECONDARY
localhost:27018 -> PRIMARY
localhost:27019 -> SECONDARY
Rejoined replica set
```

Test 7: Double Failure

Stop two nodes now (including PRIMARY)

```
localhost:27017 -> SECONDARY
localhost:27018 -> PRIMARY
localhost:27019 -> SECONDARY
localhost:27017 -> SECONDARY
localhost:27018 -> PRIMARY
localhost:27019 -> SECONDARY
localhost:27017 -> PRIMARY
localhost:27018 -> SECONDARY
localhost:27019 -> SECONDARY
localhost:27017 -> PRIMARY
localhost:27018 -> SECONDARY
localhost:27019 -> SECONDARY
Replica set unreachable
Replica set unreachable
Replica set unreachable
Replica set unreachable
Replica set unreachable
Replica set unreachable
```

Result: Double failure correctly caused loss of quorum.